

CryptoAdvisor AI — 项目设计文档

版本: 1.0 | 日期: 2026-04-04

目录

- [1. 项目概述](#)
- [2. 核心功能](#)
- [3. 设计理念](#)
- [4. 技术栈](#)
- [5. 亮点: LLM Function Calling 与 LangChain](#)
- [6. 系统架构](#)
- [7. 数据流与 AI 分析流程](#)
- [8. 多数据源聚合策略](#)
- [9. 安全设计](#)
- [10. UI/UX 设计系统](#)

1. 项目概述

CryptoAdvisor AI 是一款面向加密货币投资者的智能辅助工具。用户选择目标币种、配置投资风格，系统自动聚合 8 个维度的实时市场数据，借助大型语言模型生成专业分析报告，并输出可直接执行的结构化交易指令，一键完成下单。

核心价值主张：

- 将繁琐的多源数据收集、技术分析、风险评估全部自动化
- 通过 LLM Function Calling 将 AI 的自然语言输出约束为可机器执行的交易 JSON
- 零后端、零数据库，所有敏感信息保留在用户本地

2. 核心功能

2.1 多币种实时行情

- 支持搜索并多选目标交易对（最多 5 个），如 `BTC/USDT`、`ETH/USDT`
- 实时获取价格、24h 涨跌幅、成交量（通过 CCXT 对接 BYDFi 交易所）
- 补充数据源 CoinPaprika 提供市值、流通量等基本面数据

2.2 投资参数配置

参数	选项
投资风格	进取型 / 稳健型 / 保守型
投资周期	短期 (< 1 月) / 中期 (1-6 月) / 长期 (> 6 月)
投资金额	自由输入, 单位 USDT
自定义偏好	最多 500 字, 直接注入 AI Prompt, 影响分析方向

2.3 市场数据面板

聚合展示全球宏观指标：

- **恐惧贪婪指数** (Alternative.me)：反映市场整体情绪
- **BTC / ETH 市值主导率** (CoinGecko)：判断资金流向
- **Binance 资金费率 & 未平仓合约**：衡量衍生品多空力量
- **BTC 链上 Mempool 状态**：评估链上活跃度与矿工费
- **DeFi TVL** (DefiLlama)：监测以太坊生态流动性
- **实时新闻** (CoinDesk)：捕捉市场突发事件

2.4 AI 流式分析报告

- 调用大语言模型（支持 Claude / GPT-4o / 智谱 GLM）生成投资分析
- 报告以 **Markdown 流式渲染**，内容实时逐字呈现，无需等待完整响应
- 报告结构：
 1. 市场综合评估
 2. 衍生品情绪分析
 3. 逐币种技术面与基本面分析
 4. 风险评估与仓位建议
 5. 关键价位（止损 / 止盈 / 入场目标）

2.5 结构化交易指令输出

AI 分析完成后，通过 **Function Calling** 输出标准 CCXT 订单数组：

```
[
  {
    "symbol": "BTC/USDT",
    "type": "limit",
    "side": "buy".
```

```
    "amount": 0.002,
    "price": 82000,
    "reasoning": "RSI 超卖区间, 支撑位附近布局",
    "params": {
      "stopLoss": 78000,
      "takeProfit": 91000
    }
  }
}
```

用户可在界面预览订单详情，确认后输入交易所 API Key，一键调用 CCXT 执行下单。

2.6 历史记录管理

- 每次分析结果（报告 + 市场快照 + 订单）自动持久化至 **IndexedDB**
- 支持按时间倒序浏览历史，查看完整报告及对应下单 JSON
- **再次使用**：点击按钮可将历史记录的所有参数回填至主界面，重新下单
- 支持单条删除和一键清空

2.7 多语言 & 主题

- 中文 / 英文双语切换，通过 Cookie + 服务端读取消除 SSR hydration 闪烁
- 深色 / 浅色主题切换，通过内联脚本 + CSS 变量实现无闪烁切换

3. 设计理念

3.1 数据驱动决策

AI 分析不依赖单一数据源，而是整合价格行情、市场情绪、衍生品持仓、链上数据、宏观新闻等 **8 个维度**，构建完整的市场信息上下文，让 LLM 在充分信息下给出更准确的判断。

3.2 结构化约束 AI 输出

纯自然语言的 AI 分析难以被程序直接消费。项目通过 **LLM Function Calling** 强制 AI 在完成文字分析后，以 JSON Schema 约束的格式提交交易指令，实现"分析报告"与"可执行指令"的双轨并行输出。

3.3 隐私优先

项目刻意设计为纯前端架构，不依赖任何自建后端：

- AI API Key 存于 `localStorage`，不离开用户设备
- 交易所凭证存于 `sessionStorage`，标签页关闭即销毁
- 分析历史存于 `IndexedDB`，数据完全本地化

3.4 性能与体验

- **流式响应**：AI 内容边生成边展示，消除长时间等待的空白感
- **多级缓存**：按数据更新频率设置差异化 TTL（1 min ~ 15 min），减少重复 API 调用
- **CORS 代理**：通过 Next.js rewrites 在服务端代理所有外部 API，消除浏览器跨域限制

3.5 视觉设计哲学

采用 **Modern Dark Glassmorphism** 风格，以金融科技交易面板为参照：深色背景降低视觉疲劳，玻璃拟态卡片赋予层次感，Orbitron 标题字体传递科技感，JetBrains Mono 数据字体保证数值清晰可读。

4. 技术栈

层级	技术	说明
框架	Next.js 16 (App Router)	SSR + API Routes，兼顾 SEO 与动态交互
语言	TypeScript 5（严格模式）	全量类型覆盖，禁用 <code>any</code>
样式	Tailwind CSS v4	Utility-first，配合 CSS 自定义属性实现主题切换
AI SDK	Anthropic SDK / OpenAI SDK	原生调用，支持流式响应与 Tool Use
LLM 集成	LangChain Core + Community	链式调用与结构化输出封装
交易所	CCXT 4.5 (BYDFi)	统一交易所接口，支持现货 & 永续合约
本地存储	IndexedDB (via <code>idb</code>)	分析历史持久化
图标	Lucide React	统一矢量图标库
字体	Orbitron + JetBrains Mono	标题 / 数据分场景使用
Markdown	react-markdown + rehype-highlight	安全渲染 AI 报告
国际化	自实现 <code>i18n</code> (Cookie + SSR)	无闪烁语言切换

5. 亮点：LLM Function Calling 与 LangChain

本节是项目最核心的技术亮点，重点介绍。

5.1 问题背景

大语言模型默认输出自然语言文本。在金融应用场景中，仅有可读性强的分析报告是不够的——我们需要 AI 同时输出两类内容：

1. **Markdown 分析报告**：供用户阅读，包含市场分析、逻辑推导、风险提示
2. **结构化交易 JSON**：供程序消费，直接传入 CCXT 执行下单

如何确保 AI 的结构化输出在格式上绝对可靠，而不是随意生成一段"看起来像 JSON"的文本？答案是 **Function Calling**（工具调用）。

5.2 Function Calling 核心机制

项目在 `/src/app/api/analyze/route.ts` 中为 LLM 定义了一个 `submit_orders` 工具：

```
// Anthropic 格式的工具定义
const tools = [{
  name: 'submit_orders',

  description: '分析报告完成后，调用此工具提交结构化 ccxt 订单数组。必须在输出完整分析报告后
再调用。',

  input_schema: {
    type: 'object',
    required: ['orders'],
    properties: {
      orders: {
        type: 'array',
        description: '建议的 ccxt 订单列表',
        items: {
          type: 'object',
          required: ['symbol', 'type', 'side', 'amount'],
          properties: {
            symbol: { type: 'string', description: '交易对，例如 BTC/USDT' },
            type: { type: 'string', enum: ['limit', 'market'] },
            side: { type: 'string', enum: ['buy', 'sell'] },
            amount: { type: 'number', description: '交易数量（币单位）' },
            price: { type: 'number', description: 'limit 单填价格，market 单为
null' },
          },
        },
      },
      reasoning: { type: 'string', description: '一句话下单理由' },
      params: {
        type: 'object',
        properties: {
          stopLoss: { type: 'number', description: '止损价（USDT）' },
          takeProfit: { type: 'number', description: '止盈价（USDT）' },
        },
      },
    },
  },
},
],
```

```
}  
}]
```

工作原理：

1. LLM 在完成文字分析后，主动调用 `submit_orders` 工具
2. 调用参数严格符合上述 JSON Schema（由 LLM Provider 在网络层保证格式正确性）
3. 服务端拦截 `tool_use` 事件，提取 `input.orders` 数组

这与普通 Prompt 要求 AI "输出 JSON" 有本质区别：Function Calling 的参数由 LLM Provider 做 Schema 验证，格式可靠性接近 100%。

5.3 三大 AI 提供商的适配

项目同时支持 Anthropic、OpenAI、智谱 GLM，三者的 Function Calling API 略有差异，项目在同一路由中做了统一适配：

```
Anthropic  → tool_use / input_json_delta  
OpenAI     → tool_calls / function / arguments (流式 JSON 拼接)  
GLM        → tool_calls (兼容 OpenAI 格式)
```

每个提供商的流式事件格式不同，项目通过分支处理后向客户端统一输出同一种格式的 `ReadableStream`，下游解析逻辑无需感知差异。

5.4 流式输出 + 结构化数据的双轨设计

Function Calling 带来了一个工程问题：如何在同一个 HTTP 流中同时传输 Markdown 文本和 JSON 订单？

项目采用了轻量级的自定义协议：在流的末尾追加一个特殊标记行：

```
... (Markdown 报告文本流) ...  
  
__ORDER_JSON__: [{"symbol": "BTC/USDT", "type": "limit", ...}]
```

客户端（`/src/lib/claude.ts`）的流解析器逻辑：

```
// 逐块读取 ReadableStream  
while (true) {  
  const { done, value } = await reader.read()  
  if (done) break  
  
  const chunk = decoder.decode(value)  
  fullText += chunk  
  
  // 检测到标记前，将内容实时推送给 UI  
  if (chunk.startsWith('__ORDER_JSON__')) {  
    // 解析 JSON 订单  
    const orders = JSON.parse(chunk.substring(14))  
    // 推送给 UI  
    ui.postMessage(orders)  
  }  
}
```

```

if (!markerSeen && !chunk.includes(ORDER_MARKER)) {
  onChunk(chunk) // 触发 UI 实时渲染
} else {
  markerSeen = true
}
}

// 流结束后，按标记分割内容与订单
const [reportContent, orderStr] = fullText.split(ORDER_MARKER)
const orders: CCXTOrder[] = JSON.parse(orderStr)

onComplete(reportContent.trim(), orders)

```

这种设计实现了：

- 报告实时展示：Markdown 内容边生成边渲染，用户不需要等待 AI 调用工具
- 订单完整交付：流结束后订单 JSON 完整解析，不存在截断风险

5.5 LangChain 的作用

项目引入 **LangChain Core** 和 **LangChain Community** 提供链式调用与结构化输出的标准化封装：

- **@langchain/core**：提供 `BaseMessage`、`HumanMessage`、`SystemMessage` 等消息抽象，统一不同 LLM 提供商的输入格式
- **@langchain/community**：提供 `ChatAnthropic`、`ChatOpenAI` 等模型适配器，将 LangChain 的标准 Chain 接口映射到各提供商原生 API
- **withStructuredOutput / StructuredOutputParser**：结合 Zod Schema 对 AI 输出做运行时类型校验，确保 `CCXTOrder[]` 数组结构严格符合 TypeScript 类型定义
- **Prompt Template**：使用 `ChatPromptTemplate.fromMessages()` 构建可复用、可插值的 Prompt 模板，将用户配置（风格、周期、金额）与市场数据动态注入

整体 **LangChain** 数据流：

```

InvestmentConfig + MarketSnapshot
    ↓
ChatPromptTemplate.fromMessages([
  SystemMessage (角色设定 + 分析框架),
  HumanMessage (市场数据 + 用户配置)
])
    ↓
ChatAnthropic / ChatOpenAI (绑定 submit_orders 工具)
    ↓
StreamingChain (流式输出 + 工具调用并行)
    ↓
[Markdown Report Stream] + [CCXTOrder[] via Function Calling]

```

5.6 Prompt 工程

Prompt 采用分层结构，确保 AI 输出质量：

【角色设定】

你是专业加密货币投资顾问，擅长技术与风险管理...

【用户画像】

投资风格: {intent} | 周期: {period} | 金额: {amount} USDT
目标币种: {symbols}
自定义偏好: {customTendency}

【实时市场数据】

现货行情: {tickers} // 价格、涨跌幅、成交量
市场情绪: {fearGreed} // 恐惧贪婪指数 (0-100)
衍生品数据: {fundingRates} // 资金费率、未平仓合约
链上数据: {btcOnchain} // Mempool 状态
宏观新闻: {news} // 最新市场资讯

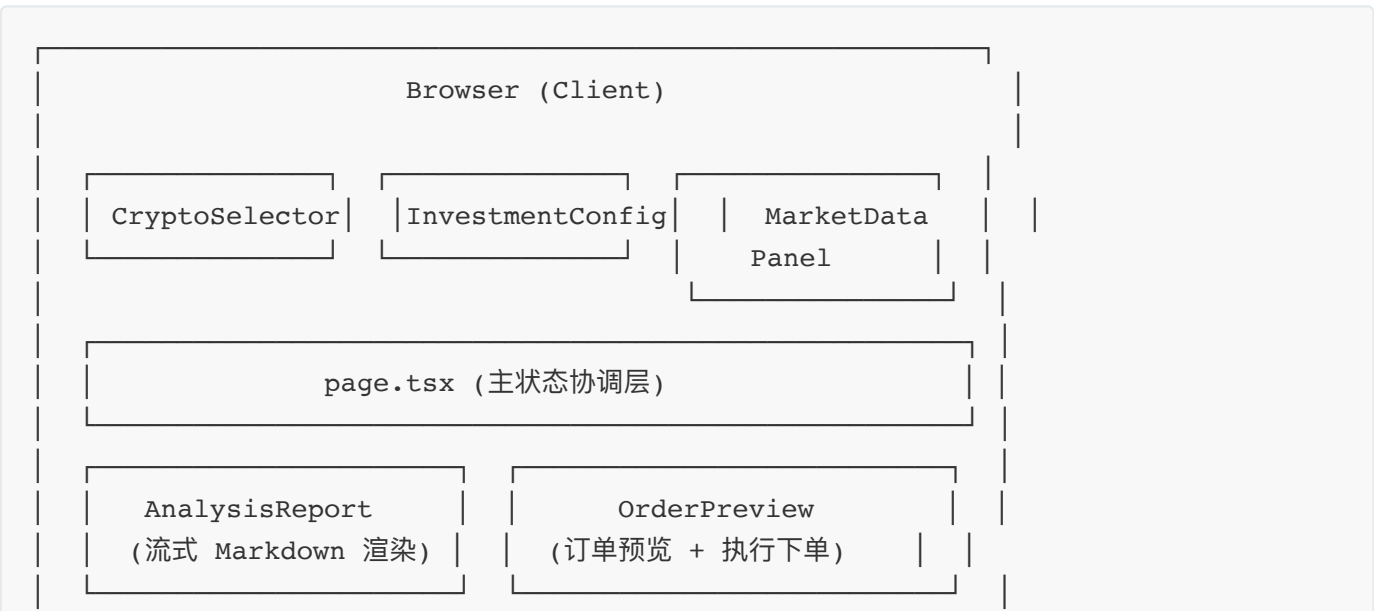
【分析要求】

1. 市场综合评估 (宏观 + 情绪)
2. 衍生品多空情绪分析
3. 逐币种技术面 + 基本面
4. 风险评估 + 仓位建议
5. 关键价位 (入场 / 止损 / 止盈)

【输出规范】

先输出完整 Markdown 报告，再调用 submit_orders 工具提交订单
订单金额合计不超过用户投资金额

6. 系统架构





7. 数据流与 AI 分析流程



```
buildPrompt()  
→ 组装详细中英文 Prompt  
  
LangChain Chain  
ChatAnthropic.bindTools([  
  submit_orders (JSON Schema)  
)  
.stream(messages)
```

ReadableStream

④ 客户端逐块接收流

```
onChunk(text) → UI 实时渲染  
  
检测 __ORDER_JSON__: 标记  
分离 Markdown 与 JSON
```

onComplete(report, orders)

⑤ 持久化

saveAnalysis(IndexedDB) → 完整记录存档

⑥ 用户确认订单

输入 BYDFi API Key/Secret (sessionStorage)

⑦ POST /api/order

CCXT BYDFi.createOrder(...)

→ 交易所执行

8. 多数据源聚合策略

8.1 数据源一览

数据源	代理前缀	提供数据	缓存 TTL
BYDFi (CCXT)	/api/market	现货实时价格	1 min
CoinPaprika	/proxy/coinpaprika	全球市值、流通量	5 min
Alternative.me	/proxy/alternative	恐惧贪婪指数	15 min
DefiLlama	/proxy/llama	以太坊 DeFi TVL	15 min

CoinGecko	/proxy/coingecko	BTC/ETH 主导率	5 min
Binance Futures	/proxy/binfutures	资金费率、未平仓	1 min
Mempool.space	/proxy/mempool	BTC 链上状态	2 min
CoinDesk	/proxy/coindesk	实时新闻	10 min

8.2 缓存机制

缓存实现在 `src/lib/apiCache.ts`，采用模块级 `Map<key, {data, expiresAt}>` 内存缓存：

- **服务端**：在 Node.js 进程生命周期内跨请求复用，减少对外部 API 的重复调用
- **客户端**：在浏览器标签页生命周期内有效

TTL 分级原则：数据波动越快，TTL 越短；高频数据（资金费率）1 分钟刷新，低频数据（新闻、恐惧贪婪）15 分钟刷新。

9. 安全设计

数据类型	存储位置	生命周期	原因
AI API Key	localStorage	永久（用户主动删除）	频繁使用，跨会话保留
交易所 API Key/Secret	sessionStorage	标签页关闭即销毁	高敏感，最短存活时间
分析历史（含市场快照）	IndexedDB	永久（用户主动删除）	完全本地，不上传服务端
语言偏好	Cookie	1 年	服务端 SSR 读取，消除 FOUC
主题偏好	localStorage	永久	纯 CSS，内联脚本读取

其他安全措施：

- **XSS 防护**：AI 报告通过 `react-markdown` 渲染，严禁 `dangerouslySetInnerHTML`
- **无服务端存储**：用户的交易凭证和分析数据均不经过任何自建服务器
- **环境变量隔离**：Anthropic API Key 通过 `NEXT_PUBLIC_ANTHROPIC_API_KEY` 注入，`.env.local` 加入 `.gitignore`

10. UI/UX 设计系统

10.1 视觉风格

Modern Dark Glassmorphism — 金融科技 / 加密货币交易面板风格。

10.2 色彩系统

Token	值	用途
<code>--bg-deep</code>	<code>#020203</code>	最深背景, OLED 优化
<code>--bg-base</code>	<code>#050506</code>	主背景
<code>--bg-elevated</code>	<code>#0a0a0c</code>	卡片/模块背景
<code>--accent</code>	<code>#5E6AD2</code>	主色调 (蓝紫)
<code>--success</code>	<code>#22C55E</code>	看涨 / 成功
<code>--danger</code>	<code>#EF4444</code>	看跌 / 危险
<code>--warning</code>	<code>#F59E0B</code>	中性 / 警告
<code>--foreground</code>	<code>#EDEDEF</code>	主文字

10.3 玻璃拟态卡片规范

```
background:      rgba(255, 255, 255, 0.05);
backdrop-filter: blur(20px);
border:          1px solid rgba(255, 255, 255, 0.08);
border-radius:   16px;
box-shadow:      0 8px 32px rgba(0, 0, 0, 0.4);
```

10.4 动画规范

- **Micro** (150ms) : 按钮 hover、图标切换
- **Standard** (250ms) : 面板展开、Tab 切换
- **Complex** (350ms) : 抽屉滑入、模态框出现
- Easing: `cubic-bezier(0.16, 1, 0.3, 1)` (弹出感)
- 必须支持 `prefers-reduced-motion`

10.5 布局规范

- 左侧边栏 (`w-72`, 可折叠至 `w-12`) : 币种选择 + 投资配置
- 中间主区域 (`min-w-0`) : 市场数据 + AI 报告
- 右侧订单面板 (`w-72 xl:w-80`) : 订单预览 + 执行

历史面板：从左侧滑入的抽屉（fixed left-0 top-0 h-full）

- 历史面板：从左侧滑入的抽屉（fixed left-0 top-0 h-full）
- 移动端：底部 Tab Bar 切换视图，最小支持宽度 375px